Fall 2026 Exploratory Data Analysis with R

Lecture 9: Regression II and III

Brian Robertson
Federal Reserve Bank of Atlanta
2026-10-30

01

Recap & Agenda

01 Recap & Agenda
02 Indicator Variables
03 Interaction Terms
04 Variable Transformations
05 Project Discussion
06 Recap

Quick Recap: Last Week

- **Linear regression** models a relationship between an outcome variable and an explanatory variable as a straight line, $y = mx + b$
- Ordinary least squares (OLS) picks the slope and intercept that minimize the sum of squared residuals
- `lm()` fits the model; `broom::tidy()` turns the results into a tidy data frame instead of a wall of `summary()` text
- A coefficient can be **statistically significant** (precisely different from zero) without being **economically significant** (large enough to matter)

Today: regression with more than one explanatory variable. We'll let different groups have their own intercept (**indicator variables**), their own slope (**interaction terms**), and we'll fit curved relationships without leaving the linear model (**transformations**).

Goals for Today

- Add a categorical variable to a regression with **indicator (dummy) variables**
- Pull a coefficient, standard error, or p-value out of a model with `broom::tidy()` — never `summary(model)$coefficients`
- Let a relationship's slope differ by group with **interaction terms**
- Fit a non-linear relationship with a **log transformation**, without leaving the linear regression framework
- Compare models with `broom::glance()`
- Connect today's tools to the final project

02

Indicator Variables

01 Recap & Agenda
02 Indicator Variables
03 Interaction Terms
04 Variable Transformations
05 Project Discussion
06 Recap

Summary Statistics by State

Before adding a variable to a regression, look at it. Our data covers home sales in DC, Maryland, and Virginia:

```
redfin_clean %>%
  group_by(state) %>%
  summarise(n = n(),
            mean_ppsf = mean(ppsf, na.rm = TRUE),
            mean_dist = mean(metro_distance, na.rm = TRUE)) %>%
  kable(digits = 2, col.names = c("State", "N", "Mean price/sq ft", "Mean metro distance"))
```

State	N	Mean price/sq ft	Mean metro distance
DC	623	451.04	0.61
MD	285	321.99	1.59
VA	252	401.07	1.14

- Average price per square foot already differs quite a bit by state — before we even account for distance to the metro.

Motivation: Indicator Variables

- What else, besides distance from a metro stop, might affect housing prices?
- What about another “location” factor — which state the home is in?
- Our baseline regression gives every sale the **same intercept**, regardless of state
- To let each state have its own intercept, we need a way to represent a categorical variable — like `state` — numerically
- The tool for this is an **indicator (dummy) variable**: a column of 0s and 1s that marks membership in a category

Indicator Variables: Building the Data

- Represent each category with its own 0/1 column
- 1 means membership in that category, 0 means not a member
- Every row belongs to exactly one category

```
redfin_clean %>%
  mutate(DC = if_else(state == "DC", 1, 0),
         MD = if_else(state == "MD", 1, 0),
         VA = if_else(state == "VA", 1, 0)) %>%
  select(ppsf, metro_distance, state, DC, MD, VA) %>%
  slice_sample(n = 5) %>%
  kable(digits = 2)
```

ppsf	metro_distance	state	DC	MD	VA
121.56	3.03	MD	0	1	0
365.48	2.45	VA	0	0	1
374.77	0.43	DC	1	0	0
170.23	0.66	MD	0	1	0
805.71	0.45	DC	1	0	0

Indicator Variables: Running the Regression

`lm()` builds these indicator columns automatically from a character or factor variable — you don't need to construct `DC`/`MD`/`VA` by hand:

```
reg_indicators <- lm(ppsf ~ metro_distance + state, data = redfin_clean)
```

- Adding `state` gives each state its own **intercept**
- All three states still share the same **slope** on `metro_distance`

Indicator Variables: Results

```
tidy(reg_indicators) %>%
  kable(digits = 3)
```

term	estimate	std.error	statistic	p.value
(Intercept)	496.696	7.413	67.002	0.000
metro_distance	−74.380	6.370	−11.676	0.000
stateMD	−56.353	12.850	−4.385	0.000
stateVA	−10.854	12.204	−0.889	0.374

- R dropped one category — `DC` — to avoid perfect collinearity between the indicators and the intercept. This dropped category is the **base group**.
- Every other coefficient is a difference **relative to the base group**.

Indicator Variables: The Model

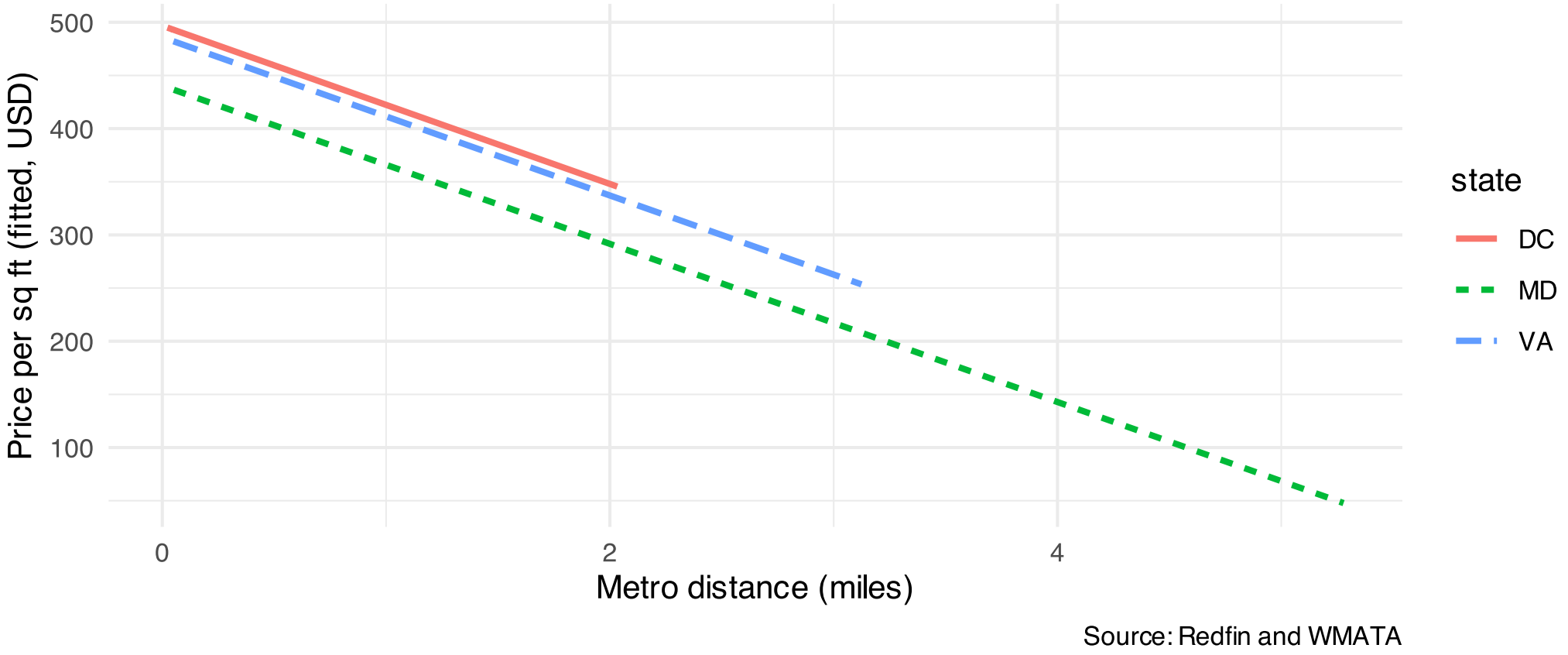
$$\text{Price/Sq Ft} = m \times \text{Distance} + b_{DC} + b_{MD} \times \text{MD} + b_{VA} \times \text{VA}$$

This single equation is really three equations, one per state:

- $\text{Price/Sq Ft}_{DC} = m \times \text{Distance} + b_{DC}$
- $\text{Price/Sq Ft}_{MD} = m \times \text{Distance} + (b_{DC} + b_{MD})$
- $\text{Price/Sq Ft}_{VA} = m \times \text{Distance} + (b_{DC} + b_{VA})$

The lines have different intercepts, but — because there's no interaction yet — the **same slope**, m.

Visualizing Different Intercepts



Source: Redfin and WMATA

- Three parallel lines — same slope, different starting points.

Interpreting the Results

```
m  <- tidy(reg_indicators) %>% filter(term == "metro_distance") %>% pull(estimate)
b_dc <- tidy(reg_indicators) %>% filter(term == "(Intercept)")  %>% pull(estimate)
b_md <- tidy(reg_indicators) %>% filter(term == "stateMD")      %>% pull(estimate)
b_va <- tidy(reg_indicators) %>% filter(term == "stateVA")      %>% pull(estimate)
```

- **Intercept (497):** the expected `ppsf` for a DC home right at a metro stop (`metro_distance` = 0). Statistically significant (p < 0.001).
- **Slope (-74):** holding state fixed, each extra mile from the metro is associated with about $74 less per square foot. Statistically significant (p < 0.001).
- **MD indicator (-56):** at the same distance from the metro, an MD home sells for about $56 less per square foot than a comparable DC home. Statistically significant (p < 0.001).
- **VA indicator (-11):** the point estimate says a VA home sells for about $11 less per square foot than DC, but this is **not** statistically significant — we can't rule out that VA and DC have the same intercept.

Economic Significance

- An average home in this sample is about 1608 square feet.
- The MD indicator alone is worth about $91K on a home of that size — clearly large enough to matter for a buyer or seller.
- Always ask both questions: is the estimate **precise** (statistically significant), and is it **large** (economically significant)? They are not the same thing.

Breakout: Indicator Variables (Page 1 of 2)

So far we've used `state` as our categorical variable. Let's try a different one: `property_type`. A few property types have very few sales, so we'll drop them first.

```
redfin_houses <- redfin_clean %>%
  filter(property_type %in% c("Condo/Co-op", "Single Family Residential", "Townhouse"))
```

1. Fit a regression of `ppsf` on `metro_distance` and `property_type` using `redfin_houses`.
2. Use `broom::tidy()` to view the results.
3. Which `property_type` is the base group? How can you tell?

Breakout: Indicator Variables (Page 2 of 2)

4. Pick one `property_type` indicator. Is it statistically significant?
5. What does that indicator's coefficient mean, in plain English, about the price of that property type relative to the base group?

Solution

```
reg_property <- lm(ppsf ~ metro_distance + property_type, data = redfin_houses)

tidy(reg_property) %>%
  kable(digits = 3)
```

term	estimate	std.error	statistic	p.value
(Intercept)	508.747	8.628	58.965	0.000
metro_distance	−84.500	5.761	−14.667	0.000
property_typeSingle Family Residential	−28.548	11.671	−2.446	0.015
property_typeTownhouse	−25.870	11.181	−2.314	0.021

- The base group is `Condo/Co-op` — `lm()` drops the first category alphabetically (by default) when it builds indicators from a character column, so neither `Single Family Residential` nor `Townhouse` appears as its own row; both are measured as a difference from `Condo/Co-op`.
- Both the `Single Family Residential` and `Townhouse` indicators are negative and statistically significant ($p < 0.05$): holding distance from the metro fixed, both property types sell for less per square foot than a comparable condo.

03

Interaction Terms

01 Recap & Agenda
02 Indicator Variables
03 Interaction Terms
04 Variable Transformations
05 Project Discussion
06 Recap

Motivation: Interaction Terms

- Indicator variables let each state have its own **intercept** — but we forced every state to share the same **slope** on `metro_distance`
- Is that realistic? Maybe the *penalty* for being far from a metro stop isn't the same in every state
- An **interaction term** is the product of two explanatory variables — here, an indicator and `metro_distance`
- Adding interaction terms lets each group have its own slope **and** its own intercept

Interaction Terms: Running the Regression

```
reg_interactions <- lm(ppsf ~ metro_distance + state * metro_distance,
                       data = redfin_clean)
```

- `state * metro_distance` tells `lm()` to include `state`, `metro_distance`, **and** their interaction, `state:metro_distance`, all at once
- This is equivalent to writing out `metro_distance + state + state:metro_distance` by hand

Interaction Terms: Results

```
tidy(reg_interactions) %>%
  kable(digits = 3)
```

term	estimate	std.error	statistic	p.value
(Intercept)	563.521	11.558	48.755	0
metro_distance	−183.246	15.941	−11.495	0
stateMD	−165.998	19.704	−8.425	0
stateVA	−82.790	20.211	−4.096	0
metro_distance:stateMD	135.777	17.945	7.566	0
metro_distance:stateVA	113.350	19.842	5.713	0

- Two new rows appear: `metro_distance:stateMD` and `metro_distance:stateVA` — these are the interaction terms

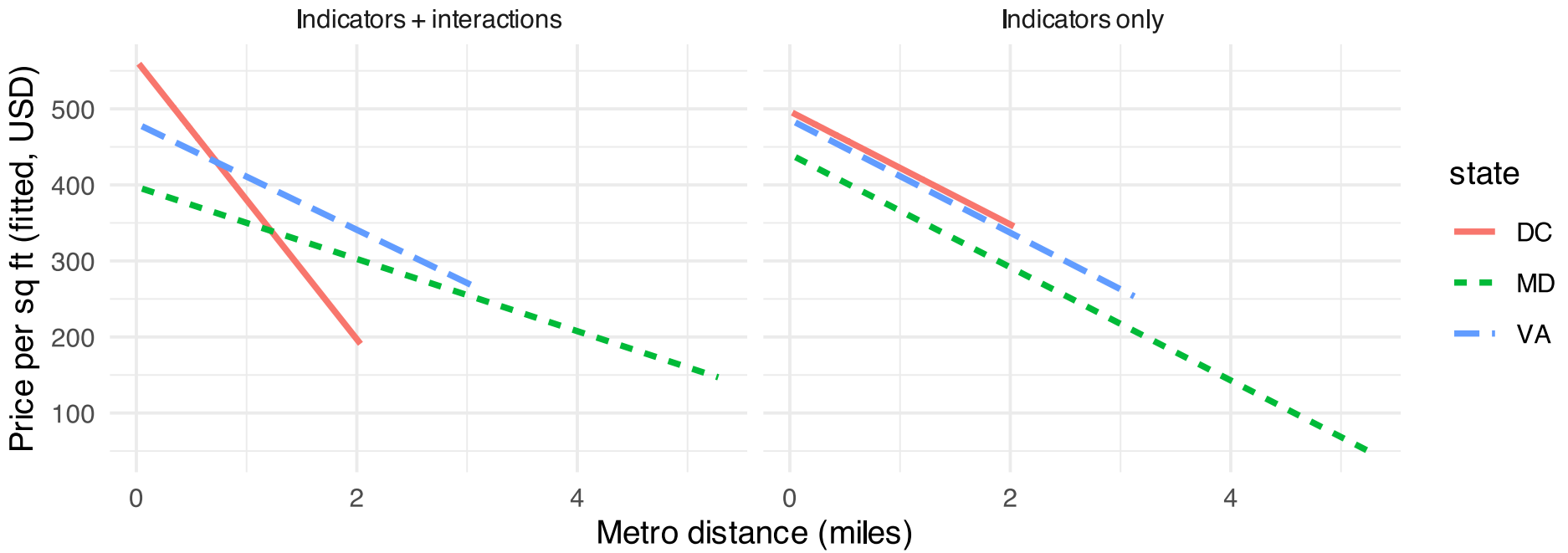
Interaction Terms: The Model

Adding `metro_distance:state` lets each state's slope (m) and intercept (b) differ, instead of only its intercept. Written out, that's really three separate equations:

- $\text{Price/Sq Ft}_{DC} = m_{DC} \times \text{Distance} + b_{DC}$
- $\text{Price/Sq Ft}_{MD} = (m_{DC} + m_{MD}) \times \text{Distance} + (b_{DC} + b_{MD})$
- $\text{Price/Sq Ft}_{VA} = (m_{DC} + m_{VA}) \times \text{Distance} + (b_{DC} + b_{VA})$

Now the lines can differ in **both** slope and intercept.

Visualizing Different Slopes and Intercepts



Source: Redfin and WMATA

- With interactions, the lines are no longer parallel — each state's price falls off at its own rate as distance increases.

Interpreting the Results

```
m_dc <- tidy(reg_interactions) %>% filter(term == "metro_distance")           %>%
pull(estimate)
m_md <- tidy(reg_interactions) %>% filter(term == "metro_distance:stateMD")   %>%
pull(estimate)
m_va <- tidy(reg_interactions) %>% filter(term == "metro_distance:stateVA")   %>%
pull(estimate)
b_dc <- tidy(reg_interactions) %>% filter(term == "(Intercept)")              %>%
pull(estimate)
b_md <- tidy(reg_interactions) %>% filter(term == "stateMD")                  %>%
pull(estimate)
b_va <- tidy(reg_interactions) %>% filter(term == "stateVA")                  %>%
pull(estimate)
```

- **DC slope (-183):** in DC, each extra mile from the metro predicts about $183 less per square foot.
- **MD interaction (136):** MD's slope is 136 *less negative* than DC's — so MD's price-per-mile-from-metro effect is $m_{DC} + m_{MD} \approx -47$. Statistically significant ($p < 0.001$): distance matters less in MD than in DC.
- **VA interaction (113):** similarly, VA's slope is $m_{DC} + m_{VA} \approx -70$. Also statistically significant.

Comparing Predicted Values by State

```
tibble(
  state = c("DC", "MD", "VA"),
  slope = c(m_dc, m_dc + m_md, m_dc + m_va),
  intercept = c(b_dc, b_dc + b_md, b_dc + b_va)
) %>%
  kable(digits = 1, col.names = c("State", "Slope", "Intercept"))
```

State	Slope	Intercept
DC	−183.2	563.5
MD	−47.5	397.5
VA	−69.9	480.7

- Compare this to the indicator-only model: there, every state had the *same* slope by construction. Interactions let the data tell us otherwise.

Breakout: Interaction Terms (Page 1 of 2)

Using `redfin_houses` from the last breakout:

1. Fit a regression of `ppsf` on `metro_distance`, `property_type`, and their interaction.
2. Use `broom::tidy()` to find the slope on `metro_distance` for the base property type.
3. Find the slope for one other property type by adding the base slope to that type's interaction coefficient.

Breakout: Interaction Terms (Page 2 of 2)

4. Compare `broom::glance()` for this model to the indicator-only model from the last breakout. Which has the higher R^2?
5. Does allowing a different slope per property type change how you'd describe the relationship between distance and price?

Solution

```
reg_property_int <- lm(ppsf ~ metro_distance + property_type * metro_distance,
                       data = redfin_houses)

tidy(reg_property_int) %>%
  kable(digits = 3)
```

term	estimate	std.error	statistic	p.value
(Intercept)	535.598	9.743	54.970	0.000
metro_distance	−118.365	8.333	−14.204	0.000
property_typeSingle Family Residential	−142.339	19.370	−7.349	0.000
property_typeTownhouse	−46.804	15.811	−2.960	0.003
metro_distance:property_typeSingle Family Residential	99.638	13.772	7.235	0.000
metro_distance:property_typeTownhouse	27.079	13.552	1.998	0.046

Solution, Continued

```
bind_rows(
  glance(reg_property)     %>% mutate(model = "Indicators only"),
  glance(reg_property_int) %>% mutate(model = "Indicators + interactions")
) %>%
  select(model, r.squared, adj.r.squared) %>%
  kable(digits = 3, col.names = c("Model", "R-squared", "Adjusted R-squared"))
```

Model	R-squared	Adjusted R-squared
Indicators only	0.184	0.182
Indicators + interactions	0.221	0.217

- The interaction model has the higher R^2 — allowing a different slope per property type explains more of the variation in `ppsf`.
- A higher R^2 alone doesn't tell us the extra complexity is worth it — we'd want the interaction coefficients themselves to be statistically and economically meaningful, which (from the table on the previous slide) they are.

Comparing All Three Models

```
bind_rows(
  glance(reg_baseline)     %>% mutate(model = "Baseline (no state)"),
  glance(reg_indicators)   %>% mutate(model = "Adds indicators"),
  glance(reg_interactions) %>% mutate(model = "Adds interactions")
) %>%
  select(model, r.squared, adj.r.squared) %>%
  kable(digits = 3, col.names = c("Model", "R-squared", "Adjusted R-squared"))
```

Model	R-squared	Adjusted R-squared
Baseline (no state)	0.175	0.174
Adds indicators	0.189	0.187
Adds interactions	0.227	0.224

- Each added layer of complexity — indicators, then interactions — raises R^2. That's expected: more free parameters can only fit the sample at least as well.
- The right question isn't just "does R^2 go up," but whether the added terms are motivated by theory and are themselves statistically and economically meaningful.

Pooled vs. Grouped Data

- Our baseline model pools all three states together and estimates one relationship
- The indicator and interaction models let the data split apart by state
- It’s possible for a pooled regression and a group-by-group analysis to point in different directions — this is sometimes called **Simpson’s Paradox**
- We’ll return to this tension when we talk about correlation and causation later in the course

Key Takeaways

- **Indicator variables** let a categorical variable shift the intercept — each group gets its own starting point, same slope
- **Interaction terms** let a categorical variable also shift the slope — each group gets its own line, start to finish
- Add either kind of term when theory, prior research, or the data itself (summary statistics, a scatterplot) suggests groups behave differently
- `broom::tidy()` and `broom::glance()` — not `summary(model)$coefficients` — are the tools for pulling numbers out of a fitted model

04

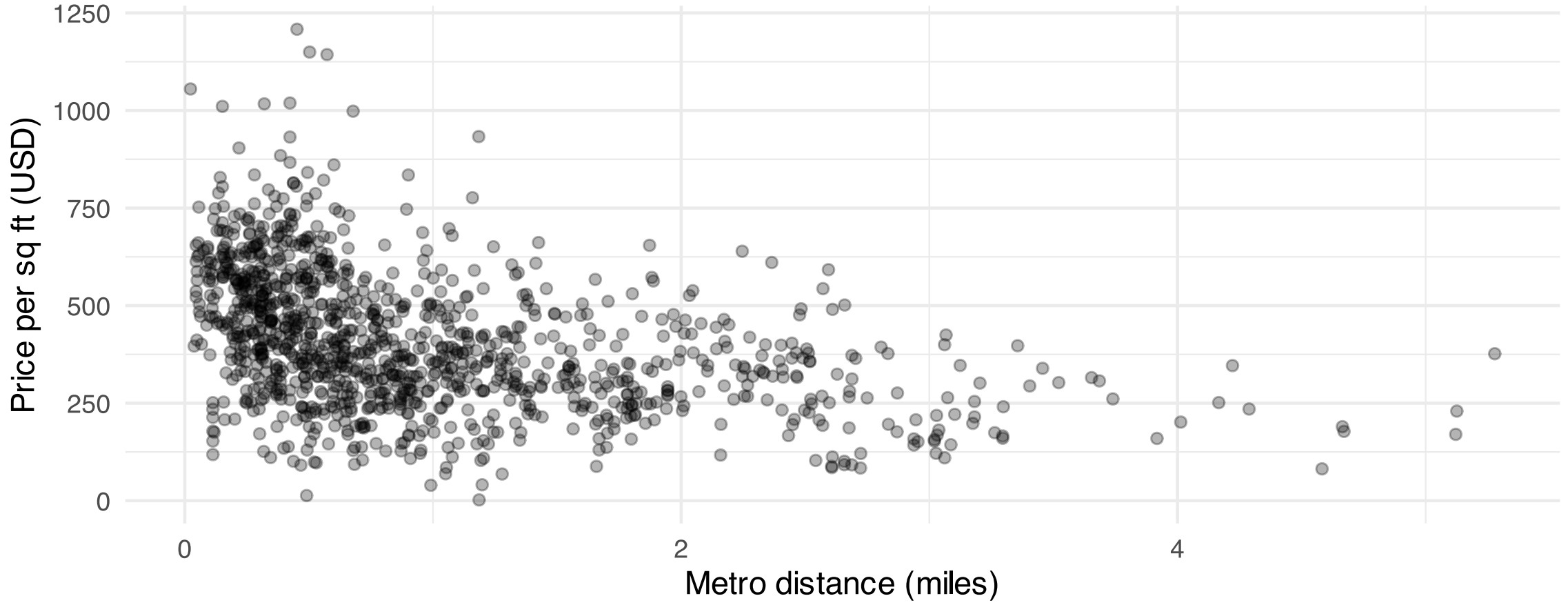
Variable Transformations

01 Recap & Agenda
02 Indicator Variables
03 Interaction Terms
04 Variable Transformations
05 Project Discussion
06 Recap

Motivation: Non-Linear Relationships

- A linear regression always fits a straight line — but not every relationship in the data looks like a straight line
- What would lead us to suspect a non-linear relationship?
  - Intuition, theory, or prior research
  - The shape of a scatterplot
- We don't have to abandon `lm()` to capture this. We can **transform** an explanatory variable — for example, take its logarithm — and still fit a linear model
- "Linear regression" means linear *in the coefficients*, not that x and y must be related by a straight line

A Possibly Non-Linear Relationship



Source: Redfin and WMATA

- The drop-off looks steep close to the metro and flatter farther away — a hint that a log transformation of `metro_distance` might fit better than a straight line.

Fitting a Log Transformation

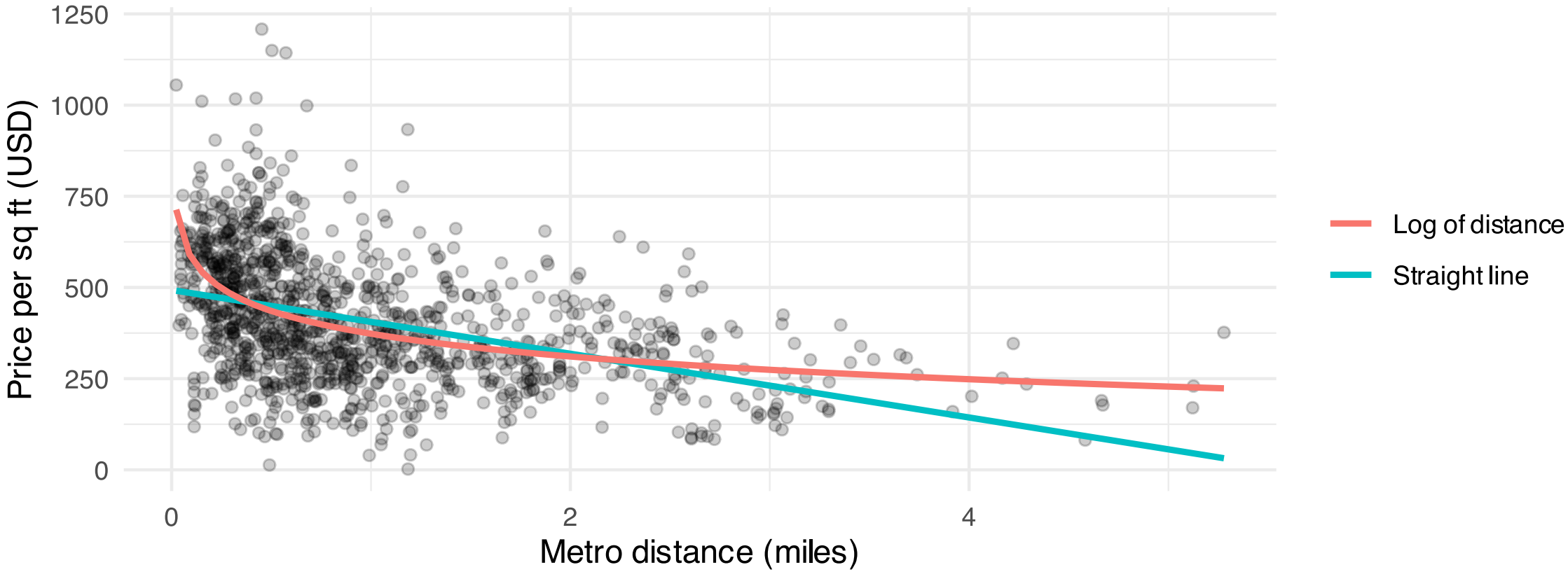
```
reg_log <- lm(ppsf ~ log(metro_distance), data = redfin_clean)

tidy(reg_log) %>%
  kable(digits = 3)
```

term	estimate	std.error	statistic	p.value
(Intercept)	373.202	4.954	75.327	0
log(metro_distance)	−90.019	5.069	−17.760	0

- `log()` goes directly inside the `lm()` formula — no need to create a new column first
- The model is still linear in its coefficients: $\text{Price/Sq Ft} = m \times \log(\text{Distance}) + b$

Comparing the Straight-Line and Log Fits



Source: Redfin and WMATA

Comparing Model Fit

```
bind_rows(
  glance(reg_baseline) %>% mutate(model = "metro_distance"),
  glance(reg_log)      %>% mutate(model = "log(metro_distance)")
) %>%
  select(model, r.squared, adj.r.squared) %>%
  kable(digits = 3, col.names = c("Explanatory variable", "R-squared", "Adjusted R-
squared"))
```

Explanatory variable	R-squared	Adjusted R-squared
metro_distance	0.175	0.174
log(metro_distance)	0.214	0.213

- The log fit explains somewhat more of the variation in `ppsf` than the straight-line fit — consistent with the bend we saw in the scatterplot.

Interpreting the Coefficient on `log(metro_distance)`

```
m_log <- tidy(reg_log) %>% filter(term == "log(metro_distance)") %>% pull(estimate)
```

- With a log-transformed x and an untransformed y, the coefficient does **not** mean "a one-unit increase in x." It means: a **1% increase** in `metro_distance` is associated with a change in `ppsf` of about $m/100$.
- $m \approx -90$, so a 1% increase in distance predicts about $0.9 less per square foot — not $90. Dividing by 100 is easy to forget, and it's a 100-fold error if you do.

- For a bigger change, the exact formula is $\Delta y \approx m \times \ln(1 + \frac{q}{100})$ for a $q\%$ change in x:

```
pct_change <- 10
m_log * log(1 + pct_change / 100)
```

```
[1] -8.579682
```

- A 10% increase in metro distance predicts about $8.58 less per square foot — using the $m/100$ shortcut (9) is a close approximation for small percentage changes, but the log formula is exact.

05

Project Discussion

01 Recap & Agenda
02 Indicator Variables
03 Interaction Terms
04 Variable Transformations
05 Project Discussion
06 Recap

Research Process: Building Toward the First Draft

The first full draft of your project is coming up. Indicator variables, interaction terms, and transformations are exactly the kind of tools a project exploring group differences or a non-linear relationship might reach for.

This Week's Timeline

- **HW #9 — due November 5.** Today's material: indicator variables, interaction terms, and log transformations.
- **First full draft of your project — due November 13 by 9am.**

We now have some time in class for you to work on your project — flag your adviser or a TA if you get stuck.

06

Recap

01 Recap & Agenda
02 Indicator Variables
03 Interaction Terms
04 Variable Transformations
05 Project Discussion
06 Recap

Recap

- **Indicator variables** turn a categorical variable into 0/1 columns, giving each group its own intercept while sharing one slope
- **Interaction terms** are the product of an indicator and another variable, giving each group its own slope too
- `lm()` builds indicators and interactions automatically from a factor or character column — `category_name` and `var1:var2` syntax, or `var1 * var2` as shorthand for both main effects and their interaction
- A **log transformation** of an explanatory variable lets `lm()` fit a non-linear relationship; the coefficient is read as "a 1% change in x" instead of "a one-unit change in x"
- `broom::tidy()` and `broom::glance()` turn model output into tidy data frames — the right tool for pulling out coefficients, standard errors, p-values, and R^2

Functions Introduced Today

- `broom::tidy()` — coefficients, standard errors, and p-values as a tidy data frame
- `broom::glance()` — one-row model summary, including R^2
- `broom::augment()` — adds fitted values and residuals back onto the original data
- `var1 * var2` in an `lm()` formula — shorthand for main effects plus their interaction
- `log()` inside an `lm()` formula — transforms a variable without a separate `mutate()` step

Documentation & Further Reading

- `?lm` — regression formulas, including `*` and `:` for interactions
- `?broom::tidy`, `?broom::glance`, `?broom::augment` — tidying model output
- https://broom.tidymodels.org/ — the broom package, with examples for many model types beyond `lm()`
- `?log` — natural log and other bases

Coming up: coding review and functional programming.